

Szymon Żyrek

Software / Systems Architect · Senior Software Engineer

Tricity, Poland
stynkdev@gmail.com
szyrek@stynk.eu
github.com/SzymonZyrek

12+ years building and debugging production systems across financial platforms, embedded/storage, cloud/distributed systems and full-stack products. I work where requirements, domain modelling, architecture, implementation, verification and operations stay connected.

TECHNICAL-AREA TAGS

SYSTEMS / ARCHITECTURE

PRODUCT / UX

RELIABILITY / SECURITY

AI / DELIVERY

CAREER · GROWTH THROUGH CHANGING CONSTRAINTS



2013-2021

Misys → Finastra

Junior → Senior → Expert Software Engineer

- Started in **TopOffice**, learning to reconstruct a heterogeneous C++ / Java EE system from behavior, failures and debugger evidence; grew from local fixes into cross-product ownership.
- Modernized **CMR persistence from DataXtend ORM to JPA** without freezing product work; built **UserManagement** and a shared auth core in C reused by C++, Java via JNI and Objective-C/GNUstep to keep identity behavior in one implementation.
- Helped turn CMR/KGR/ALM capabilities into an API-driven platform: **OpenAPI contracts → shared identity/config → Docker Compose integration → MisysBoard MVP → FusionFabric**.

INTEGRATION

PLATFORM ARCHITECTURE

IDENTITY / SECURITY

PHASES



2021-2022

Intel

Firmware Engineer

- Worked near the hardware/software boundary on **NAND/storage-driver software and automated verification**, where timing, state and failure assumptions are observable rather than abstract.
- Internal test/tooling work reinforced a habit that stayed with me: treat verification infrastructure as a product, not disposable scaffolding.

EMBEDDED / STORAGE

VERIFICATION

PHASES



2022-2023

Ciklum / EverC

Full Stack Engineer · MerchantView

- Fraud/risk product work in **Groovy/Grails** during an organizational transition, combining application development with stabilization and hands-on production ownership.
- AWS, Terraform, Kubernetes, Grafana, alerting and on-call** made the operational feedback loop part of everyday engineering rather than a separate platform concern.

RISK PRODUCT

CLOUD / PLATFORM

OPERATIONS

PHASES



2023-2024

Nordea

Full-stack Developer

- Built a **Java + Angular financing capability** inside a larger microservice/microfrontend banking ecosystem.
- The useful architecture lesson was organizational: independent teams can differ internally when **contracts, security/quality gates, dependencies and release expectations** are explicit and enforceable.

BANKING PRODUCT

DISTRIBUTED CONTRACTS

SECURITY / QUALITY

PHASES

The thread so far

Legacy debugging taught me to reconstruct systems I did not yet understand; platform integration moved the focus to stable boundaries; hardware and production operations made verification and runtime behavior concrete. The next roles add organizational contracts, distributed integration and finally full product ownership.

CAREER · DISTRIBUTED SYSTEMS → END-TO-END OWNERSHIP

2024–2025

Hapag-Lloyd

Full Stack Engineer

Java/Jakarta EE systems in shipping/logistics, including event-driven integration, operational data and platform services.

Worked with asynchronous events, replicated/configuration state and independently evolving services - another reminder that distributed systems are largely about **contracts, ownership and failure handling**.

EVENT-DRIVEN

INTEGRATION

FAILURE HANDLING

PHASES

2025–present

Independent / consulting · Stynk CRM

End-to-end product & system ownership

Started from a deceptively small request - **capture data from a paper contract** - and let real use expose the domain. The system expanded into a role-aware operating layer for sales, contracts, jobs, planning, files, payments, commissions, notifications, audit and the public website.

Own the full loop: **requirements and domain modelling → UX → Angular/Django/PostgreSQL implementation → tests/CI → Docker/nginx/Linux deployment → production support**. The repository is now roughly 130k LOC with ~1,600 tests.

Repeated pricing/service variation exposed an ownership mistake in hard-coded rules. The current architecture moves toward **typed dynamic models, Studio-authored configuration and versioned pricing semantics** so the platform owns execution guarantees while the business can own more of the model it changes.

DISCOVERY / DOMAIN / UX

ARCHITECTURE

PRODUCTION OWNERSHIP

AGENT-ASSISTED DELIVERY

ALL PHASES

Stynk case study ↗ visual tour ↗

CURRENT R&D · EXTENDING THE SAME OWNERSHIP QUESTION INTO AGENTIC ENGINEERING

HackaTeam + VibeGuard

active R&D / prototypes · not presented as production products

Execution side — HackaTeam. GitHub-native agent loops, documentation-first context, CI/evidence feedback, local/hosted execution and state-driven work selection. The goal is not “LLM writes code”; it is to make delegated engineering observable and bounded.

AGENTS / DELIVERY

WORKFLOW ARCHITECTURE

EVIDENCE / BOUNDARIES

Ownership side — VibeGuard. A Tech Owner control-loop experiment: routine evidence and policy below the boundary; explicit human authority for architecture, security, irreversible semantics, exceptions and decision memory above it.

HackaTeam ↗ VibeGuard case study ↗

TECHNOLOGY RANGE

Java / Jakarta EE / Spring

C / C++

Objective-C / GNUstep

Python / Django

TypeScript / Angular / React

Groovy / Grails

PostgreSQL / JPA

OpenAPI / REST

OAuth / OIDC

Docker / Kubernetes

Terraform

Linux

Azure / AWS / GCP

GitHub Actions

MCP / local LLM tooling

EDUCATION & EVIDENCE

University of Gdansk

Bachelor's degree · Computer Science · 2012–2015

Portfolio includes commercial engineering history, Stynk and VibeGuard case studies, ML experiments and older public “engineering archaeology” projects showing how I learn abstractions by rebuilding them.

github.com/SzymonZyrek ↗

CV · Szymon Żyrek

02 / 02 · contracts → product ownership → agentic systems